

Programació d'Arduino per blocs amb Tinkercad

Introducció

Programar és l'art de donar instruccions detallades i ordenades a una màquina perquè faci exactament el que tu vols. Per programar l'Arduino necessitem un IDE de programació on escriure el codi i des d'on gravar-lo al nostre Arduino. Aquest IDE de programació serà Tinkercad i ho farem amb programació per blocs. Tinkercad permet simular circuits i programar-los sense por a cremar cap component real. És com un "laboratori virtual"!

1. Què és la programació per blocs?

És un llenguatge visual on encaixes peces de colors (com si fos un LEGO). Cada color representa una funció:

- Sortida (Blau): Per controlar [actuadors](#) (encendre LEDs, moure motors).
- Entrada (Lila): Per llegir [sensors](#) (saber si un polsador està premut).
- Control (Taronja): Per crear esperes (delays), bucles o condicions (Si... llavors...).
- Matemàtiques (Verd): Per fer operacions o comparacions (Major que, igual a...).

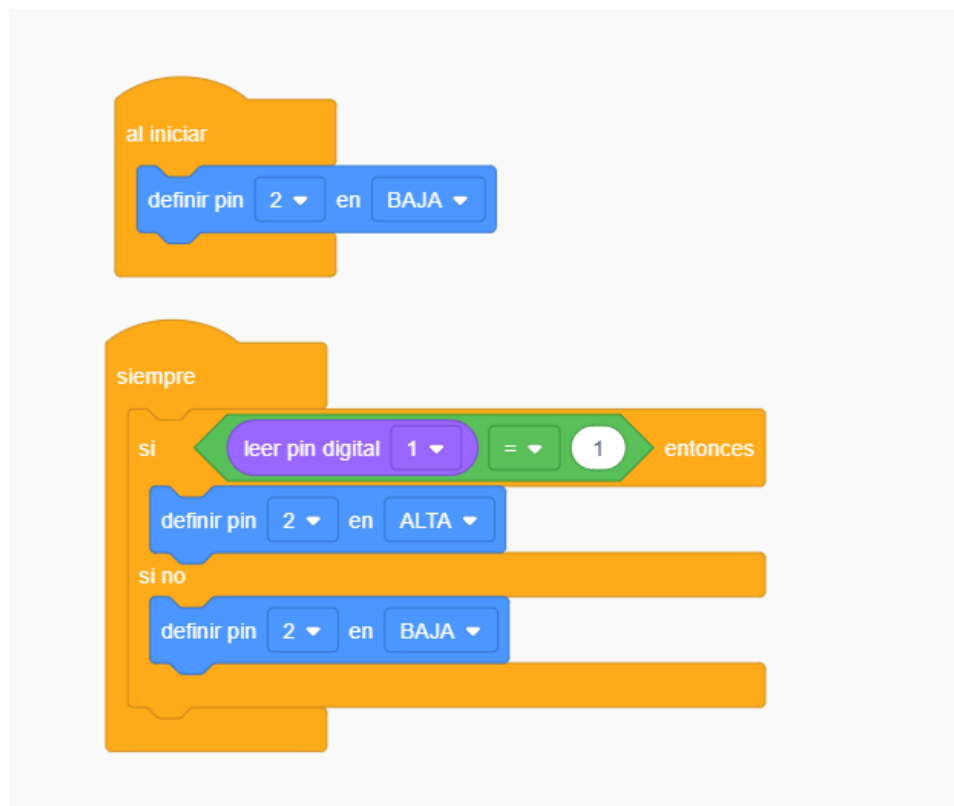


Figura 1. Exemple programa de blocs (si el PIN 1 està a nivell ALT, aleshores activem el PIN 2 a nivell ALT, de lo contrari el PIN 2 permaneceix a nivell BAJA).

2. Com començar a Tinkercad

1. Entra a Tinkercad i crea un projecte de "Circuits".
2. Busca la placa Arduino Uno a la dreta i arrossega-la a l'espai de treball.
3. Fes clic al botó "Código" (a la part superior dreta). S'obrirà el panell de blocs.

3. Conceptes clau per programar

Perquè el teu circuit funcioni, has d'entendre aquests dos blocs fonamentals:

A. Al iniciar

Aquest bloc s'empra per aplicar condicions inicials al nostre sistema (per exemple totes les eixides "apagades" o potser "enceses", o els valors d'entrada a 0, dependrà de les necessitats del programa en concret).

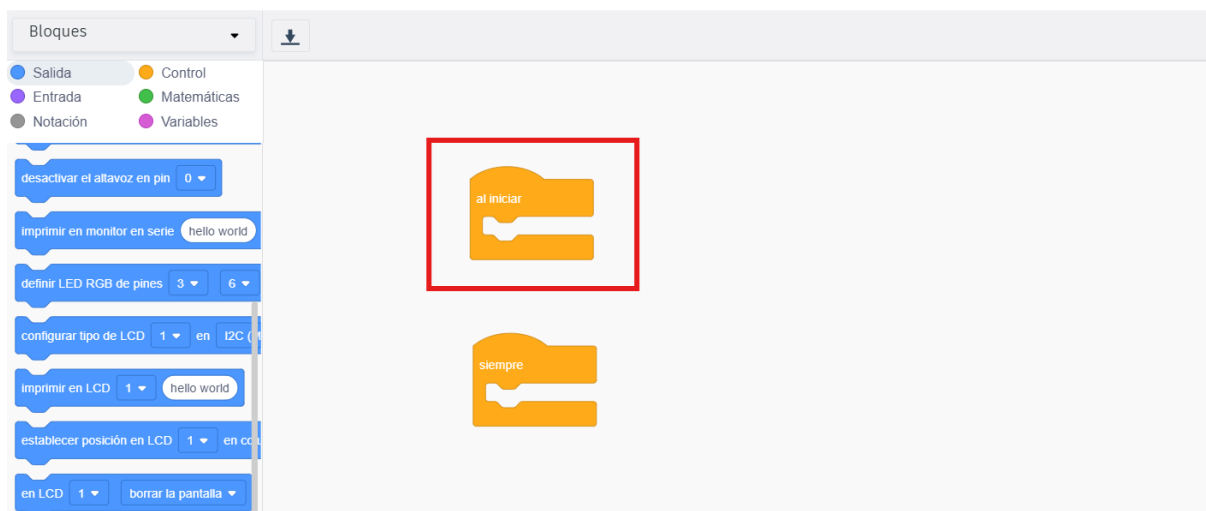


Figura 2. Bloc "al iniciar" o també conegut com Setup(). Per inicialitzar les variables del teu sistema de control. Es el estat inicial en el que estroben les eixides quan comença l'execució.

B. El bucle principal (Loop)

El codi d'Arduino s'executa en un bucle infinit. Quan arriba a l'últim bloc de baix, torna a començar pel primer de dalt instantàniament.

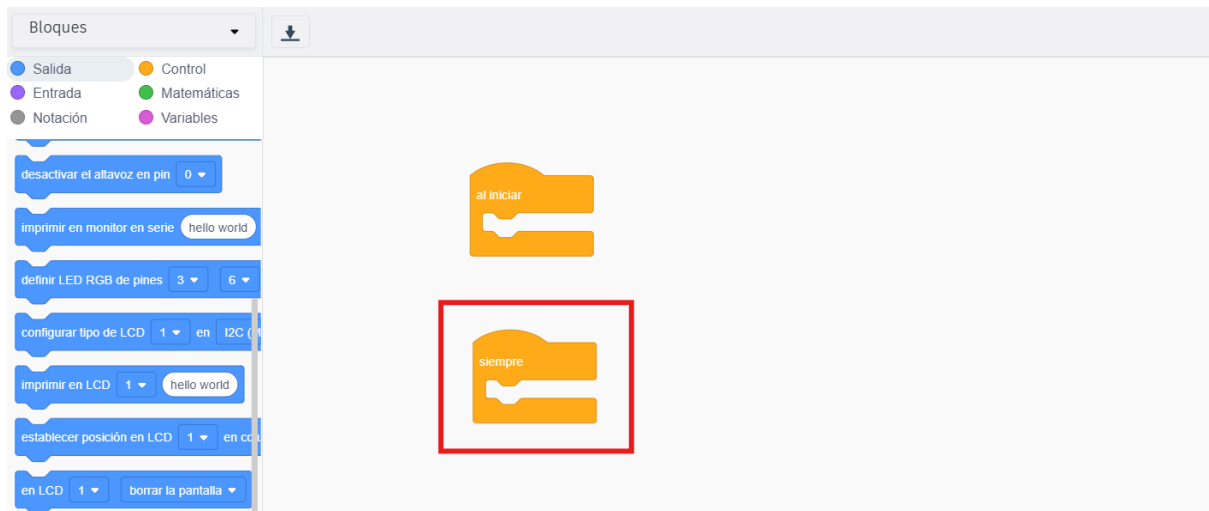


Figura 3. Bloc “siempre” o també conegut com Loop().

4. Blocs d'eixida (color blau)

Els blocs d'eixida són les ordres que envien electricitat o dades des de la placa Arduino cap als [actuadors](#). Són els que fan que "passin coses" en el món real.

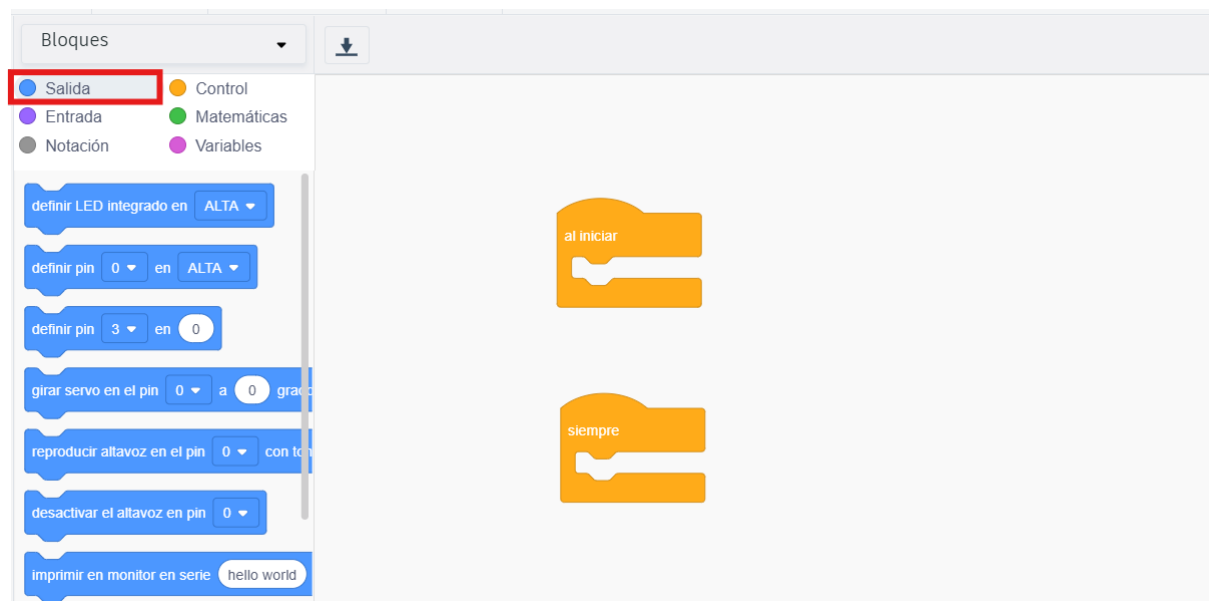


Figura 4. Blocs d'eixida.

4.1 Definir pas (Set Pin)

Aquest és el bloc fonamental per a qualsevol projecte. Serveix per controlar els pins digitals (del 0 al 13). Té dos estats:

- ALTA (HIGH): Envia 5 volts al pin. S'utilitza per encendre un LED, fer sonar un bronzidor o activar un motor.
- BAIXA (LOW): Envia 0 volts. S'utilitza per apagar el component.

4.2 Definir passador (Set Pin - PWM)

Alguns pins de l'Arduino (els que tenen el símbol ~) poden enviar valors intermedis entre 0 i 255.

- Per a què serveix? Per graduar la intensitat. Per exemple, per fer que un LED brilli només a la meitat de la seva potència o perquè un motor giri més a poc a poc.

4.3 Girar servomotor (Rotate Servo)

Aquest bloc és específic per als servomotors. En lloc de dir-li "encén-te" o "apaga't", li indiques un angle exacte (per exemple, de 0° a 180°).

- Exemple: Pots programar que una barrera s'aixequi a 90° quan passi un cotxe i torni a 0° quan hagi passat.

4.4 Reproduir a l'altaveu (Play Speaker)

S'utilitza per als bronzidors (buzzers). Et permet triar la nota musical (o freqüència) i la durada del so. És ideal per crear alarmes amb tons diferents o petites melodies.

4.5 Imprimir en el monitor en sèrie (Print to Serial Monitor)

Aquest bloc és una mica especial. No activa un component físic, sinó que envia text a la pantalla de l'ordinador.

- Per a què és útil? Per "espia" què està pensant l'Arduino. Per exemple, pots fer que t'escrigui la temperatura que està llegint un sensor per saber si el sistema funciona bé. També pots escriure comentaris per veure si el codi s'executa segons lo previst.

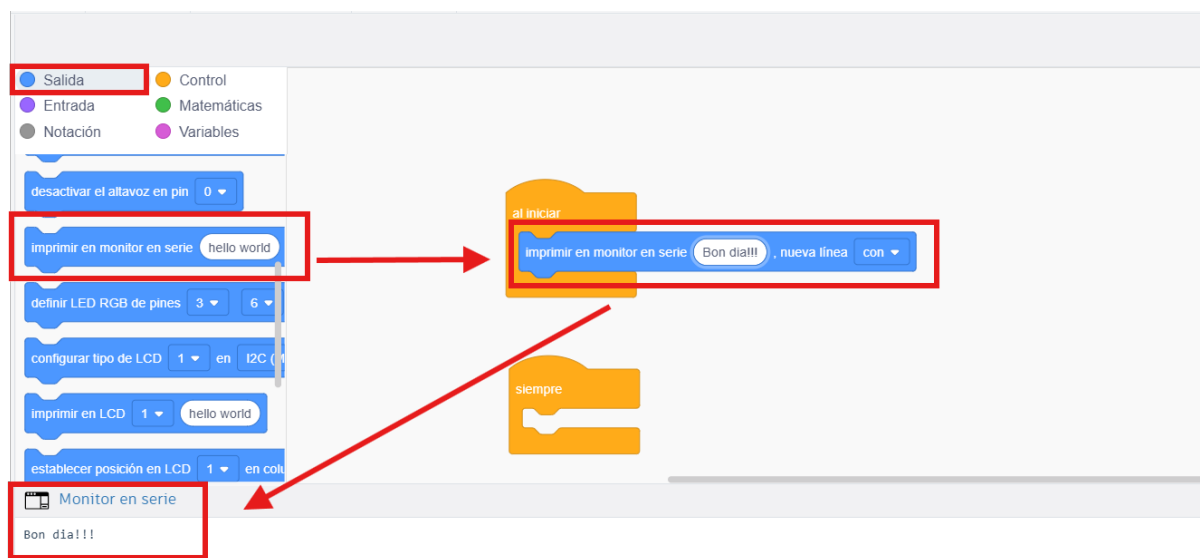


Figura 5. Bloc per imprimir al monitor serie.

5. Blocs d'entrada (color morat)

Aquests blocs llegeixen el que està passant als sensors i passen aquesta informació al controlador.

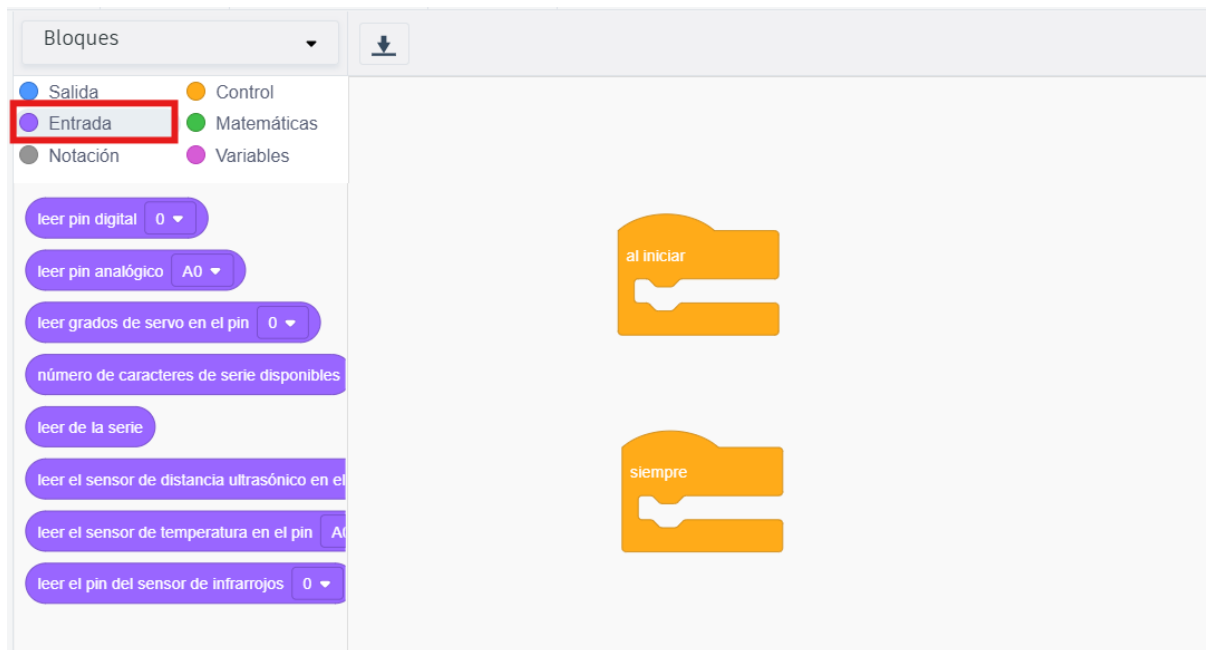


Figura 6. Bloc d'entrades.

N'hi ha de dos tipus principals:

5.1 Llegir passador digital (Read Digital Pin)

S'utilitza per a components que només tenen dos estats possibles: 0 o 1 (Fals o Cert / Baix o Alt).

- Exemple típic: Un pulsador. El bloc ens dirà si el botó està premut (1) o no (0).
- Aplicació: "Si el botó està pulsat (Entrada), encén el llum (Sortida)".

5.2 Llegir passador analògic (Read Analog Pin)

S'utilitza per a [sensors](#) que donen un rang de valors, no només "sí o no". A l'Arduino, els pins analògics (A0-A5) tradueixen el voltatge en un número entre 0 i 1023.

- Exemple típic: Un sensor de llum (LDR) o un sensor de temperatura.
- Aplicació: "Llegeix quanta llum hi ha (Entrada). Si el valor és menor de 300, vol dir que és de nit".

5.3 Blocs de sensors específics

Tinkercad inclou blocs "pre-cuinats" per a [sensors](#) molt comuns que faciliten molt la feina a l'alumne:

- Llegir sensor d'ultrasons: Et dona directament la distància en centímetres o polzades. No cal fer càlculs matemàtics complexos; el bloc ja ho fa per tu.
- Llegir sensor de temperatura: Et torna el valor directament en graus Celsius.
- Llegir sensor PIR: Et diu si ha detectat moviment o no.

6. Instruccions de control

Els blocs de Control (Taronges) són el director d'orquestra que diu qui actua i en quin moment.

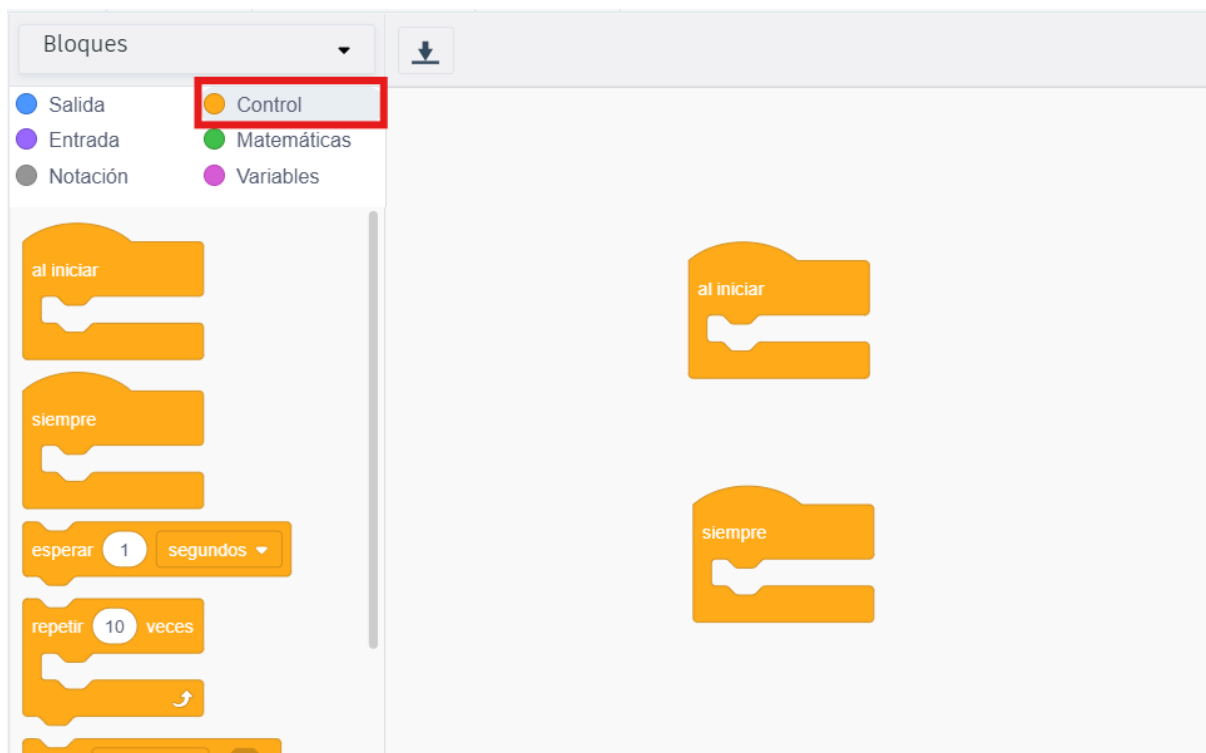


Figura 7. Bloc de control.

A Tinkercad (i en programació en general), els blocs de control més importants fan tres funcions principals:

6.1 Gestionar el Temps (Esperar / Wait)

Serveixen per aturar l'execució del programa durant uns mil·lisegons o segons.

- Per què serveixen? Són vitals per poder veure els canvis (com el parpelleig d'un llum) o per donar temps als [sensors](#) a prendre una mesura abans de la següent.

6.2 Prendre Decisions (Si... llavors... / If... then...)

Són els blocs que fan que el sistema sigui "intel·ligent". Permeten que el programa triï entre dos camins segons si es compleix una condició.

- Exemple: "Si el sensor de temperatura marca més de 30 graus, llavors encén el ventilador".
- També existeix la variant "Si... llavors... si no", que permet definir què fer quan la condició no es compleix (per exemple, apagar el ventilador si fa fred).

6.3 Repeticions i Bucles (Repetir / Loops)

Aquests blocs s'encarreguen de repetir una sèrie d'instruccions.

- Repetir X vegades: Per a accions que sabem exactament quantes vegades s'han de fer.
- Repetir mentre...: El codi s'executa una vegada i una altra fins que una condició canvia.
- El Bucle Principal (Loop): Recorda que a Arduino, tot el que posem dins l'espai de treball principal ja es repeteix per defecte infinitament.

7. Matemàtiques (color verd)

Els blocs de matemàtiques són els que permeten al sistema "pensar" i comparar dades. Són el complement indispensable dels blocs d'entrada.

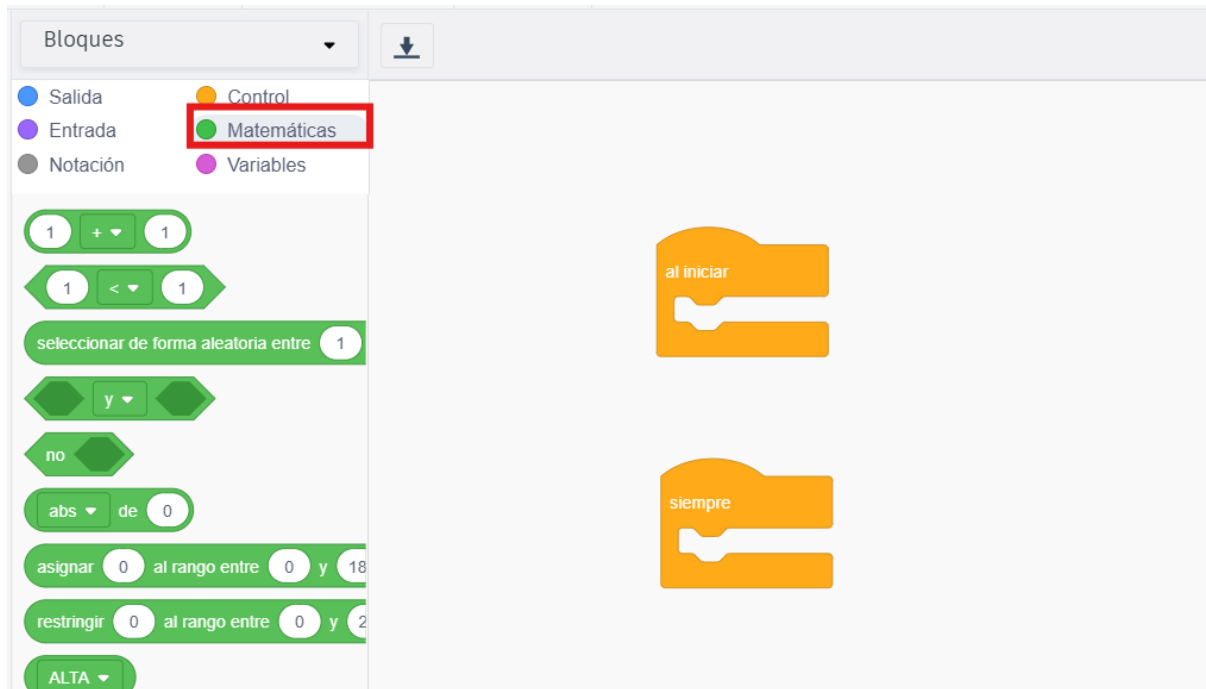


Figura 8. Bloc de matemàtiques.

7.1 Comparadors (Lògica booleana)

Són els blocs amb símbols com < (menor), > (major), = (igual) o ≠ (diferent).

- Què fan? Comparen dos valors i tornen una resposta de "Cert" o "Fals".
- Exemple: [Llegir sensor d'ultrasons] < [20]. Si l'objecte està a 15 cm, el resultat d'aquest bloc verd serà "Cert".

7.2 Operacions Aritmètiques

Són els blocs clàssics de suma (+), resta (-), multiplicació (x) i divisió (/).

- Per a què serveixen? De vegades necessites modificar la dada d'un sensor. Per exemple, si un sensor et dona la temperatura en una escala estranya, pots sumar-li o restar-li un valor per ajustar-la (això s'anomena "calibrar").

7.3 El bloc "I / O" (Operadors lògics)

Aquest bloc permet combinar dues condicions alhora.

- I (AND): Només és cert si es compleixen les dues coses (ex: "Si és de nit I a més hi ha moviment").
- O (OR): És cert si es compleix qualsevol de les dues (ex: "Si premo el botó A O premo el botó B").

8. Variables (color rosa o granat)

Els blocs de variables són fonamentals quan el teu programa necessita "recordar" una dada per utilitzar-la més tard, com podria ser un càlcul realitzat arran de 2 magnituds enregistrades per [sensors](#).

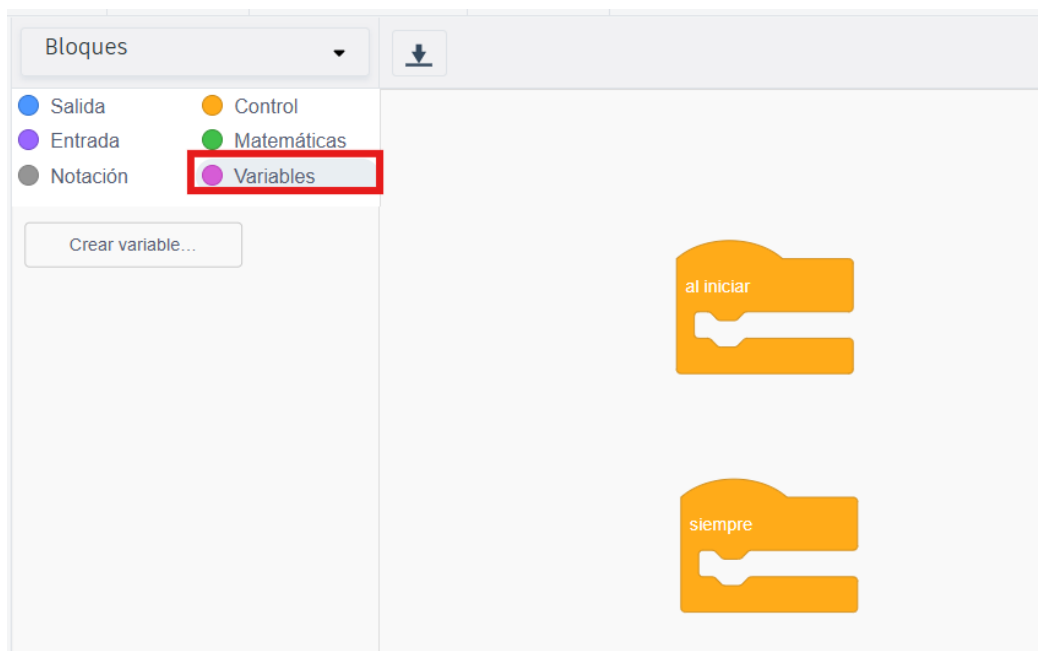


Figura 9. Bloc de variables.

Què és una variable?

Imagina que una variable és una caixa amb un nom. Dins d'aquesta caixa pots guardar un número o un valor. Durant el programa, pots obrir la caixa per veure què hi ha, o pots canviar el contingut per un número nou.

Per a què serveixen a Arduino?

1. Emmagatzemar lectures de [sensors](#): En lloc de llegir el sensor cada vegada, guardes el valor en una variable anomenada llum o distancia.
2. Comptadors: Per exemple, per comptar quantes persones han passat per una porta (persones = persones + 1).
3. Simplificar el codi: Si fas servir el Pin 13 per a un LED, pots crear una variable anomenada led_vermell amb el valor 13. Si després canvies el cable al Pin 7, només has de canviar el número a la variable i no a tot el programa.

Els 3 blocs principals de variables a Tinkercad

1. Crear variable

És el primer pas. Has de posar-li un nom clar (sense espais i millor sense accents, com valor_sensor o comptador).

2. Definir variable (Set variable)

Aquest bloc s'utilitza per guardar o canviar el valor.

- Exemple: Definir [distancia] en [Llegir sensor d'ultrasons]. Ara, la "caixa" anomenada distancia té guardats els centímetres que ha mesurat el sensor.

3. El bloc de la Variable (el "rodonet")

Un cop creada, apareix un bloc petit amb el nom de la teva variable. Aquest bloc el pots encaixar dins d'altres blocs (de sortida, de matemàtiques, etc.).

- Exemple: Si [distancia] < [10] llavors...

Un exemple pràctic: "El comptador de pulsacions"

Si volguessis comptar quantes vegades s'ha premut un botó:

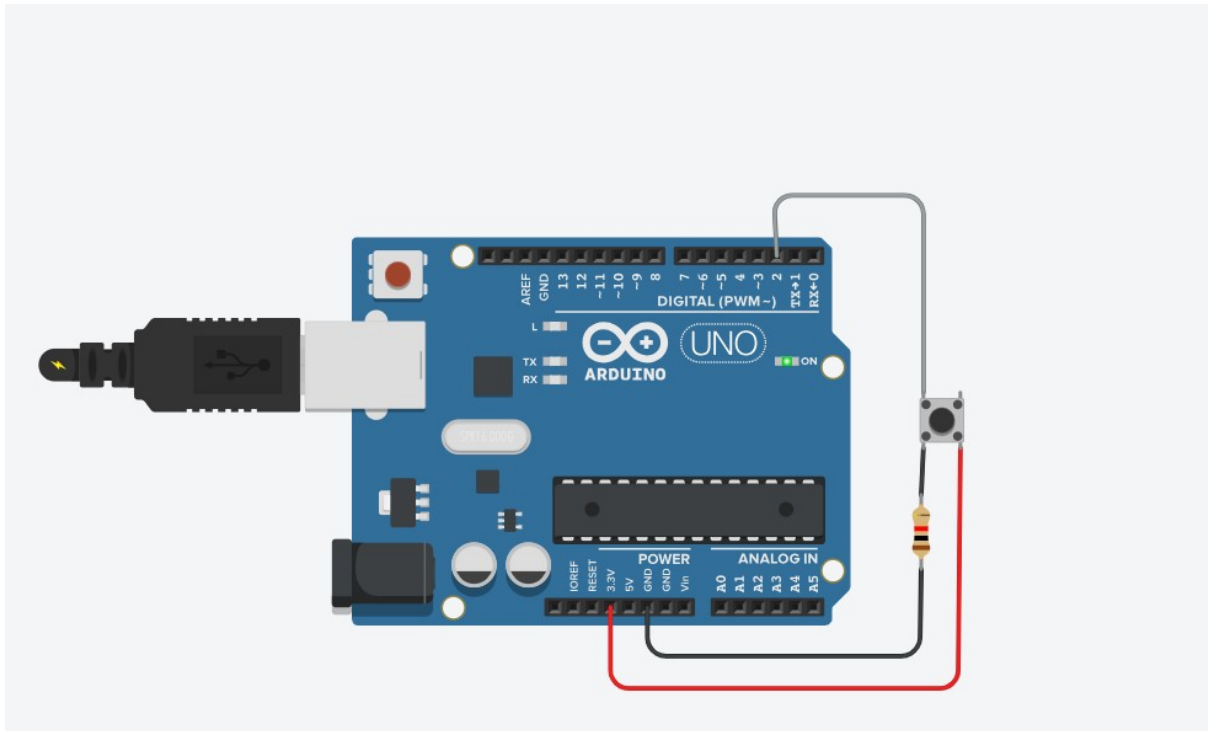


Figura 10. Circuit control de polsacions.

1. Crees la variable polsacions.

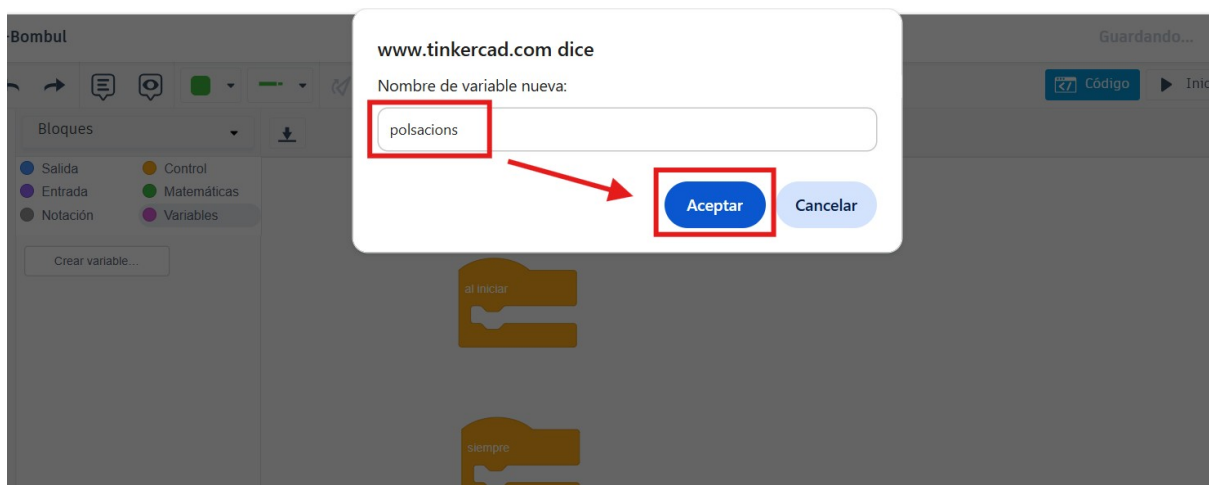


Figura 11. Declaració de la variable polsacions.

2. Al principi (Setup), la defineixes en 0.

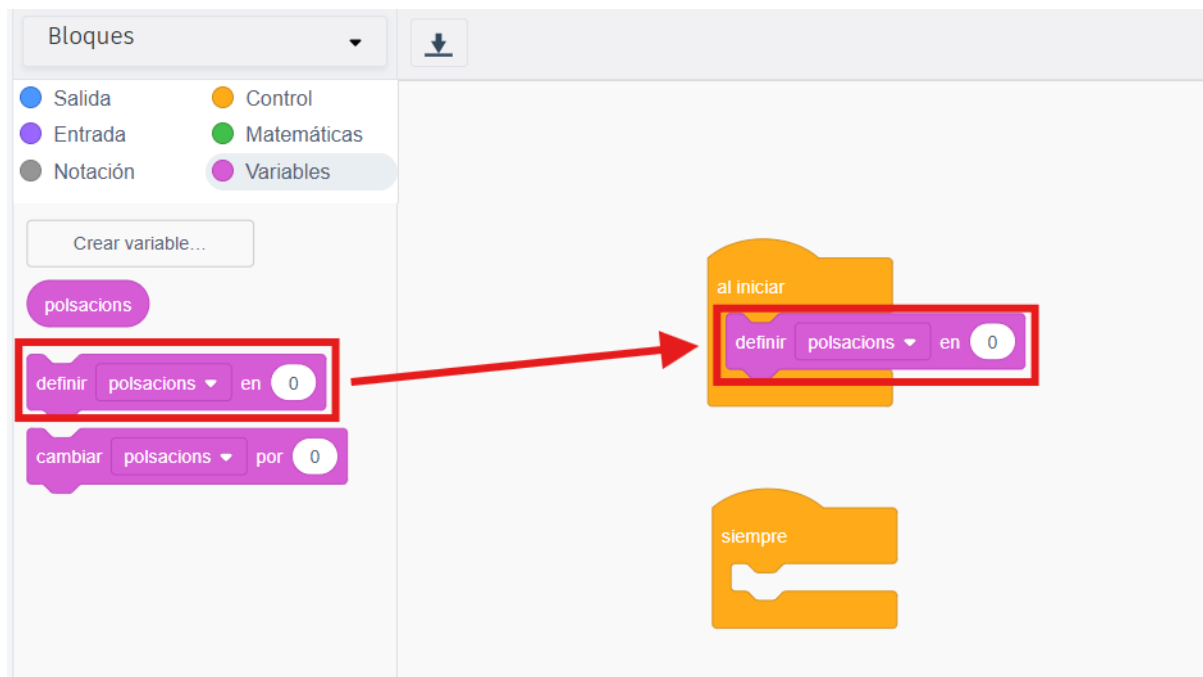


Figura 12. Inicialització del valor de pulsacions a zero.

3. SI [Llegir passador digital 2 = 1] LLAVORS:
 - Definir [pulsacions] en [pulsacions + 1].
 - Imprimir en monitor sèrie [pulsacions].

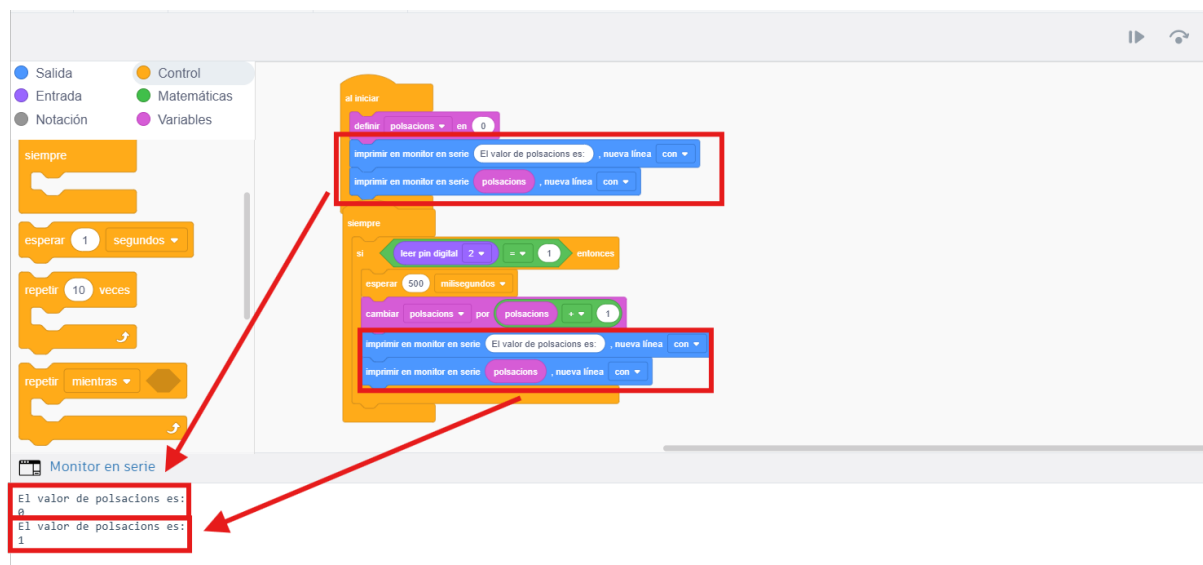


Figura 13. Resultat al monitor serie després d'iniciar l'execució del codi i haver pulsat una vegada el pulsador.

La segona línia del monitor serie s'ha produït després de la 1^a pulsació.

9. Bloc de d'anotacions (color gris)

Tot i que no fan que l'Arduino "fagi res" físicament, són una de les eines més importants per a un bon programador.

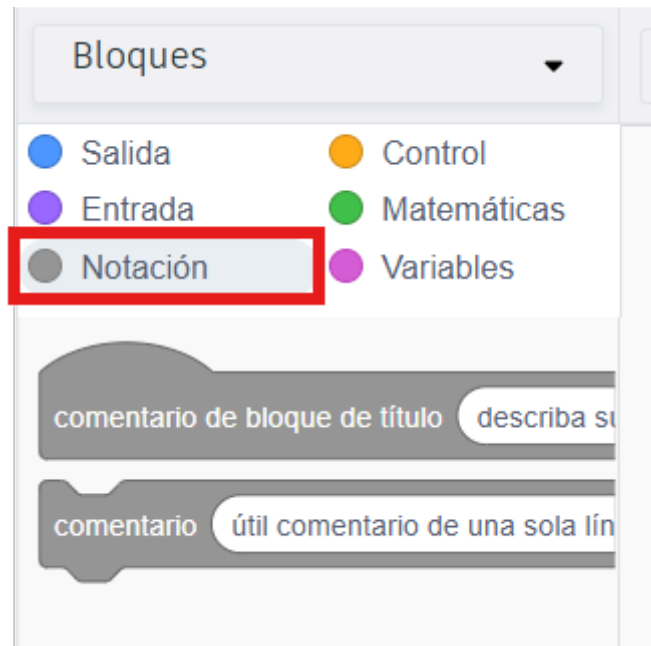


Figura 14. Bloc d'anotacions.

Què són els blocs d'Anotació?

Són blocs on pots escriure text que el microcontrolador ignora completament. Serveixen exclusivament per a la persona que llegeix el codi.

Per a què serveixen? (Les 3 utilitats clau)

1. Explicar la lògica: Si el teu programa és llarg, pots posar una anotació que digui: "Aquest grup de blocs controla el semàfor de vianants". Així, si ho llegeixes d'aquí a un mes, sabràs què estaves fent.
2. Organitzar el treball en equip: Si treballes amb un company a classe de Tecnologia, pots deixar-li notes com: "He creat la variable 'distància', però falta programar el brunzidor".
3. Depuració (Debug): Pots fer servir anotacions per "anul·lar" temporalment una part del codi sense haver d'esborrar els blocs, simplement movent-los sota una nota explicativa.

La Regla d'Or del Programador

"Programa sempre com si la persona que hagués de llegir el teu codi d'aquí a un any fos un desconegut que no sap res del teu projecte."

Com es veu això en el codi real (Text)?

Si passes de blocs a text a Tinkercad, veuràs que les teves anotacions grises es converteixen en línies que comencen amb dues barres: //.

- Bloc gris: "Llegir el sensor de llum" -> Codi: // Llegir el sensor de llum